Thai Poetry Rhyme Verification Using a Hybrid Rule-Based Approach for Education and GenAI

Warit Yuvaniyama, Athit Phinyachok, Gunn Sanguankittipun, Leo Colombo, and Prachya Boonkwan
School of ICT, Sirindhorn International Institute of Technology, Thammasat University
Pathum Thani 12120, Thailand
Email: {6622770459, 6622780474, 6622780136, 6622780540}@g.siit.tu.ac.th, prachya@siit.tu.ac.th

***Abstract*—In Thai Klon-Paed (กลอนแปด) poetry (the eight-syllable verse form), each four-line stanza must satisfy an interlocking rhyming pattern where the rhyming syllables must share both a vowel sound (sàrà สระ) and a final-consonant class (mâ:ttra: มาตรา ตัว สะกด). Because Thai script lacks word boundaries and often conceals vowels or silenced letters, automatic rhyme verification is challenging. Neither classical rule-based checkers nor G2P-romanization checkers handle the resulting edge cases reliably. We present a hybrid rule-based verification approach: first, we improve the KhaveeVerifier algorithm—vowel analysis, final-consonant class analysis, the rhyme test, karun silencing, and true-final detection—and merge it into PyThaiNLP main (Model B). Second, we introduce Klon8Checker (Model D), which augments these rules with a gold-standard G2P override dictionary of over 4,000 entries and a fallback segmenter, addressing phonetic blind spots of the base oracle. We evaluate five checker configurations on a gold corpus of 36,474 classical stanzas (145,702 rhyme checks) and on a targeted augmentation of 11,623 instances: 10,000 adversarial negatives, 1,200 tricky positives, and 423 oracle-blind rhymes. The best overall configuration (D_ssg) reaches 89.2% gold recall and 86.9% F1 on the augmentation; D_w2p attains the highest augmentation F1 (87.5%) and recovers 95.7% of the oracle-blind rhymes, while Model B achieves 100% precision on curated negatives. The verifier is released as an explainable educational tool and provides a deterministic reward signal for RL-based Klon-Paed generation.**

***Index Terms*—Thai NLP, Thai poetry, Klon-Paed, rhyme verification, rule-based systems, grapheme-to-phoneme, reinforcement learning, educational technology**

I. Introduction

Thai orthography does not map straightforwardly onto pronunciation. This gap becomes especially apparent in the classical register used by Thai poetry. The script is written without explicit word boundaries; vowels are frequently implicit; consonants or entire clusters can be silenced by the karun diacritic (t^{h}ant^{h}ák^{h}â:t ทัณฑฆาต); and irregular words whose pronunciations do not follow the standard phonetic rules are common. Complex phonological structures, such as consonant blending (*k^{h}am k^{h}uâp klâm* คำควบกล้ำ) and leading consonants (*ʔàksɔ̌n nam* อักษร นำ), further obscure syllable boundaries, vowel classification, and tone. As a result, syllabification and pronunciation are particularly difficult for computational algorithms and machine learning models.

Klon-Paed is a Thai verse form that depends on these phonetic properties. Each stanza (*bòt* บท) consists of four sets of seven-to-nine-syllable lines (*vák* วรรค) (wak). A valid rhyme requires two syllables to share both the vowel sound (sàrà สระ) (sara) and the final-consonant class (mâ:ttra: มาตราตัวสะกด). Four canonical rhyme rules must hold within (R_1, R_2, R_3) and across (R_X) stanzas (Section II), and the final syllable of every wak must participate as depicted in Fig. 1. A single silent letter, e.g. the silent *r (ร)* of *phet (เพชร)* ('diamond'), or a short-long vowel-length error is enough to break a rhyme. These constraints are strict and deterministic, which makes them difficult for statistical systems.

A reliable automatic verifier is needed in two communities. In *education*, teachers and students need deterministic, explainable, hallucination-free feedback: an LLM that invents an incorrect rhyme should not be used in a classroom. In *NLP research*, training a language model to compose Klon-Paed, e.g. via reinforcement learning (RL), requires a programmatic reward that scores rhyme correctness. Because LLMs are probabilistic, they are unreliable when handling strict constraints such as syllable counts and rhymes, which a rule-based verifier supplies deterministically.

Existing verification systems do not meet this standard. Of the five checker configurations we evaluate (A, B, C, D_w2p, D_ssg), the prior KhaveeVerifier in PyThaiNLP 5.0.1 (Model A) relies on dictionary subword tokenization, omits one canonical rhyme rule, and reaches only 74.0% recall on our gold corpus. G2P romanization-based checkers, such as the `word_check` grader of KlonSuphap-LM [1] (Model C), collapse long/short vowel distinctions and produce thousands of false positives on adversarial negatives. In this paper, we address these limitations with a hybrid rule-based approach and a rigorous, adversarial evaluation. Our contributions are:

- **An improved rule-based verifier (Model B).** We rework the vowel analysis (`check_sara`), the final-consonant-class analysis (`check_marttra`), the rhyme test (`is_sumpus`), karun silencing, and true-final detection, and implement all four canonical rhyme rules. The result is merged into PyThaiNLP main (August 4, 2026) and is set to ship as the default `KhaveeVerifier` in PyThaiNLP 6.0.
- **A dictionary-enhanced variant (Klon8Checker, Model D).** We augment the rules with a gold-standard G2P override dictionary of over 4,000 entries and a fallback segmenter, recovering the oracle's blind spots.
- **A large evaluation corpus.** 36,474 gold stanzas (145,702

rhyme checks) from five classical works, plus 11,623 augmentation instances: 10,000 oracle-verified negatives, 1,200 tricky positives, and 423 oracle-blind positives.
- **A systematic comparison.** Five configurations (A, B, C, D_w2p, D_ssg) are compared on precision, recall, F1, and coverage, with per-rule and per-operator error analyses.
- **Educational tool and RL-ready reward.** An explainable web tool plus a deterministic reward signal for RL-based Klon-Paed generation.

II. Background and Related Work

A. The Linguistic Rules of Klon-Paed

A Klon-Paed stanza has four waks of seven-to-nine syllables each. Thai syllables are written without delimiters and a word may consist of one or more syllables, so the meter must be recovered by syllabification rather than by counting words. Within this skeleton the verse follows a fixed rhyme scheme. As defined by the Royal Institute of Thailand, two syllables rhyme (*sumpus* สัมผัส) when they share the same vowel sound (sàrà สระ) and the same final-consonant class (mâːttraː มาตราตัวสะกด). Additionally, tone is not part of the rhyme. The mattra classes (*mêː* แม่), consisting of ka (กา), kong (กง), kok (กก), kot (กด), kop (กบ), kon (กน), kom (กม), koey (เกย), and koew (เกอว), group the final consonants of Thai syllables by their phonetic outcome (open syllable, velar ŋ and k, dental t, labial p, nasal n and m, semivowel j and w). The four waks are called the sàdàp (สดับ), ráp (รับ), rɔːng (รอง), and sòng (ส่ง), respectively. Four canonical rhyme rules are checked in this work, following the *klon* rhyme scheme and the implementation of the improved `KhaveeVerifier`. In rules R_1 and R_3, the target may be any of the first five syllables of the target wak, with syllables 3 and 5 as the preferred positions and syllables 1, 2, and 4 as permissible variants:

- R_1 (sàdàp → ráp): the final syllable of wak 1 must rhyme with one of the first five syllables of wak 2;
- R_2 (ráp → rɔːng): the final syllable of wak 2 must rhyme with the final syllable of wak 3;
- R_3 (rɔːng → sòng): the final syllable of wak 3 must rhyme with one of the first five syllables of wak 4;
- R_X (inter-stanza): the final syllable of wak 4 of the previous stanza rhymes with the final syllable of wak 2 of the current stanza.

As a concrete example, the well-known stanza of the *ʔhukao θhong* travel poem (นิราศภูเขาทอง) by Sunthorn Phu satisfies all rhyme rules stated above; Fig. 1 shows its rhyme positions (R_1–R_3) and the inter-stanza (R_X) link to the preceding stanza.

Three orthographic phenomena make these rules hard to apply automatically. First, compound (สระประสม), transformed (สระเปลี่ยนรูป), and reduced (สระลดรูป) vowels let several spellings map to one vowel sound. Second, the karun diacritic silences a letter or a whole cluster, leaving unpronounced letters in the written form. Third, a trailing y, l, w (ย, ล, ว) may be a genuine final consonant or merely part of a vowel digraph or an initial cluster. Any verifier must resolve all three before it can compare vowel and final class.

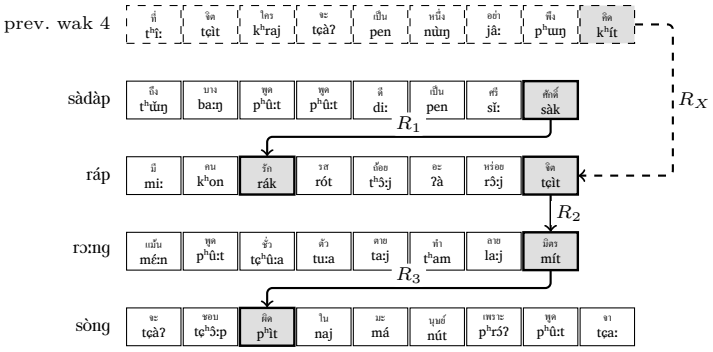


Fig. 1. The example stanza with its rhyme positions shaded and linked: R_1 (sàdàp→ráp), R_2 (ráp→rɔːng), R_3 (rɔːng→sòng); the dashed R_X arrow links the final syllable (kʰít) of the previous stanza's wak 4 to the final syllable of wak 2 (tɕìt).

B. Thai Text-Processing Foundations

Two building blocks underlie all Thai NLP systems: segmentation and grapheme-to-phoneme (G2P) conversion. Because Thai has no word delimiters, segmentation has received sustained attention: dictionary-based maximum matching (PyThaiNLP's `newmm` [2]), neural segmenters such as DeepCut [3] and AttaCut [4], syllable-based neural segmentation [5], and the SSG syllable segmenter used throughout this work. G2P maps surface orthography to pronunciation, addressed with rule-based (TLTK [6]), example-based [7], CRF joint-sequence [8], and two-stage sequence [9] models. G2P is directly relevant to rhyme verification because rhyme is a property of *sound*, not of *spelling*.

C. Computational Poetry Analysis and Generation

Prior Thai-poetry NLP work concentrates on analysis and generation. On the analysis side, machine-learning studies of Sunthorn Phu's verse extracted sound patterns such as rhyme and alliteration using internal-rhyme rules [10]. On the generation side, the dominant line fine-tunes language models on classical verse: Phra Aphai Mani LM [11] and its successor KlonSuphap-LM [1] fine-tune a GPT-2 Thai base model, then use rhyme tagging, attention masking, and reinforcement learning in which a rule-based *klon grader* supplies the reward. That grader is the `word_check` module evaluated here as Model C, making our evaluation directly relevant to that pipeline's RL step. More broadly, RLHF established reward models as essential for aligning LLMs to strict requirements [12], and grammar-constrained decoding enforces structural constraints at inference time [13].

D. Positioning of This Work

Our contribution is the first systematic, adversarially augmented evaluation of Thai rhyme *verification* as opposed to *generation*: Model A has never been stress-tested against curated edge cases, the G2P-based grader (Model C) has not been validated for precision, and no open verifier covers all four rhyme rules with dictionary support for irregular pronunciations. We fill this gap with improved rule-based verification (Model B), a dictionary-enhanced variant (Model D),

Algorithm 1 Rhyme verification of a Klon-Paed poem (`check_klon`).

Require: poem T with waks separated by whitespace; type $k \in \{4, 8\}$
Ensure: list of rhyme errors (empty $\Leftrightarrow$ poem is valid)

$W \leftarrow$ split T into waks, grouped into stanzas of four
$w_{\text{prev},4} \leftarrow$ NULL
for each stanza (w_1, w_2, w_3, w_4) **do**
  $S_i \leftarrow \text{SSG}(w_i)$, $i = 1, \ldots, 4$ {syllables}
  for each rule $\rho \in \{R_1, R_2, R_3, R_X\}$ **do**
    if $\rho = R_X$ **and** $w_{\text{prev},4} =$ NULL **then**
      continue {no previous stanza; skip R_X}
    end if
    $(a, B_\rho) \leftarrow$ source syllable and target set of rule ρ
    if no $b \in B_\rho$ satisfies IS_SUMPUS(a, b) **then**
      record an error for rule ρ
    end if
  end for
  $w_{\text{prev},4} \leftarrow w_4$ {persist for next R_X}
end for
return the list of errors

and a large gold-with-augmentation evaluation, including a novel *oracle-blind* probe that holds the verifier accountable for rhymes its oracle cannot see.

III. Methodology: Hybrid Verification Architecture

A. Core Rule-Based Verification (Model B)

Model B is the `KhaveeVerifier` merged into PyThaiNLP main and scheduled to ship in 6.0. It verifies a poem in three stages. (1) Each wak is tokenized directly into syllables with the SSG CRF segmenter, applied to the wak text itself without word pre-segmentation. (2) For each rhyme rule the relevant syllables are extracted: the final syllable of every wak and the first five syllables of waks 2 and 4. (3) The rhyme test `is_sumpus` decides whether two syllables rhyme; a rule holds if the test succeeds for at least one permitted target. Algorithm 1 summarizes this flow. The `check_klon` entry point splits the poem on whitespace, groups four waks into a stanza (rejecting incomplete stanzas), enforces per-wak word-count limits on each line (ten for Klon 8, five for Klon 4), and reports each violation as a structured message naming the stanza, wak, and offending syllables that failed to rhyme.

The verifier's accuracy depends on three core functions. `check_sara` returns the correct vowel of a syllable. Its per-character loop was rewritten to fix the condition shadowing that misparsed the ua (อัว) diphthong and to resolve the transformed and reduced vowels signaled by the short-vowel mark (*máːɪ tàɪ kʰúː* ไม้ไต่คู้ (◌็)): tɕèt (เจ็ด) → transformed e (เ-ะ), kʰěŋ (แข็ง) → transformed ɛ (แ-ะ), kɔ̀ (กี่) → transformed ɔ (เ-าะ). The *rɔː hăn* (รร) is evaluated once, yielding ʔam in a *kom class* context (e.g., *tʰam* (ธรรม)) and '*a*' elsewhere. The three pronunciations of *rú́* (ฤ) and *lú́* (ฦ) are classified from the original spelling *before* karun stripping, so silent suffixes cannot hide them: (i) *rɔ̂ːk* (ฤกษ์ - เริก), (ii) *rít* (ฤทธิ์ - ริด) and *ʔaŋkrìt* (อังกฤษ - อัง.กิด), and (iii) *rúdùː* (ฤดู - รึ.ดู). Pali/Sanskrit words with a silent terminal vowel (e.g., *kiàːt* (เกียรติ), *tɕʰâːt* (ชาติ), *tʰammátɕʰâːt* (ธรรมชาติ)) have it stripped, and an empty result falls back to the implied short '*a*'. `check_marttra` returns the Royal-Institute final-consonant class of a syllable, strictly by orthography. It resolves karun first, strips silent terminal vowels, and strips a silent *r* in loanword clusters: always in *tr* (ตร) / *chr* (ชร) / *pr* (ปร) (e.g., *but* (บัตร), *neːt* (เนตร), *pʰét* (เพชร)) but in *kr* (กร), *kh-r* (ขร), and *th-r* (ทร) only when the preceding vowel is short (*tɕàk* (จักร) loses its '*r*' while *maŋkɔːn* (มังกร) does not). For *pʰét* (เพชร) the vowel length is still misread by `check_sara` (long eː instead of short e), so *pʰét* remains an oracle-blind case (Section V). Standalone characters (*ná* (ณ), *tʰá* (ธ), *pháná* (พณ), *bɔː* (บ)) and the sàrà kɔːn are classed as *ka class*. Finally, `is_sumpus` normalizes both arguments (the old verifier had a normalization logic error): *a* (-ะ) + *koey* (เกย) → *ai* (ไ-/ใ-) + *ka* (กา) and *a* (-ะ)/*am* (ำ) + *kom* class (กม) → *am* (ำ) + *ka* class (กา). It returns true iff the normalized sara and mattra of the two syllables are equal.

Two further mechanisms account for most of the improvement over 5.0.1. `handle_karun_sound_silence` strips characters silenced by the karun, no longer by a fixed truncation: it recognizes multi-letter silent suffixes such as *pʰrá lák* (พระลักษมณ์) (exposing the true *kok class* final) and *kàsàt* (กษัตริย์) (leaving a *kot class* final), two-consonant suffixes such as *-tr* in *sàːt* (ศาสตร์) and *-ntʰr* in *tɕan* (จันทร์), and the consonant-plus-vowel patterns of *sìt* (สิทธิ์) and *pʰan* (พันธุ์), while leaving an internal karun (e.g., *ʔoːm* (โอห์ม)) untouched. `_is_true_final` decides whether a trailing *y*, *l*, or *w* is a genuine final rather than part of a vowel digraph or initial cluster. It strips karun and tone marks first (so *klâɪ* (ใกล้) is still seen as ending in *l* (ล)), requires at least two consonants, and consults hand-built cluster tables for *l*, *r*, and *w* under the pre-posed vowels (*eː*, *ɛː*, *oː*, *aɪ*, *aɪ* เ-แ-ไ-ใ-). These distinguish the cases: the *y* of *tʰaɪ* (ไทย) is silent, the *l* of *pleː* (เปล) belongs to the cluster, and the *l* of *tɕʰon* (ชล) is a true final.

Table I summarizes the principal changes of the 6.0 verifier relative to 5.0.1; they were developed and merged in the PyThaiNLP project [2]. The fixes target precisely the failure families probed by the augmentation (Section V): karun and *ru* traps, short/ long and compound-vowel swaps, and the one-sided special-vowel normalization that our C9 operator exploits. Together they raise per-rule gold recall from 86–89% to 95–97% (6.0) and eliminate false positives on the curated negatives.

B. Dictionary-Enhanced Model (Klon8Checker, Model D)

Model B is linguistically principled but inherits a blind spot from its own design: `check_sara` returns only the first vowel of a token, so a multi-syllable word that SSG keeps as one token loses the vowel of its final (rhyming) syllable, and silent-*r* loanwords are stripped only in a whitelisted subset of clusters. Klon8Checker (Model D) addresses this by layering a gold-standard pronunciation dictionary on top of Model B's rules.

TABLE I
PRINCIPAL CHANGES OF THE 6.0 KHAVEEVERIFIER OVER THE 5.0.1 BASELINE.

Aspect	5.0.1 → 6.0
Segmentation	dict subword → SSG syllable segmentation
Karun (tʰantʰákʰâːt)	naive → improved rule-based
True finals y/l/w	naive → cluster-aware (tʰai, klai→ka class)
Vowel normalization	one-sided → both-sided
Compound/ reduced vowels	misread → compound-vowel classifier, short-vowel mark, and 'ua' diphthong
rúa	one sound → three sounds (*rôːk*, *rít*, *rúa*)
R_3 rule	absent → implemented
Silent r	misread → stripped in `check_marttra` (but, pʰét†)
Single-char words	crash → guarded (na, tha→'a')
Silent terminal vowels	misread → stripped (*kìat*, *tɕʰàt*)
Stanza processing	flat → stanza-based, word counts, structured errors

†The strip fixes the final-consonant class; `check_sara` still reads the vowel as long eː rather than short e (Section V).

The dictionary, `POETRY_OVERRIDES`, contains over 4,000 hand-verified entries mapping a surface form to its syllable sequence (e.g., *pʰét* (เพชร) → pʰét; *sàttruː* (ศัตรู) → sàt-truː; *kàsàtriː* (กษัตรี) → kà-sàt-triː). It was built from the frequent vocabulary of the *pʰrá ʔàpʰaɪ mániː* (พระอภัยมณี) corpus with manual supervision, and every entry is checked by a four-guard pipeline that rejects pronunciations hallucinated by the Word2Phrase (w2p) neural G2P model; w2p-derived candidates are kept only as labeled hints and never trusted on their own. At inference time Klon8Checker segments the wak into words with an override-augmented newmm dictionary, looks each up in the override dictionary, and syllabifies unmatched words with the fallback segmenter. We report two configurations: `D_w2p`, which falls back to the w2p neural G2P model, and `D_ssg`, which falls back to plain SSG segmentation; as Section V shows, the w2p fallback hallucinates on short or compound words, making `D_ssg` the more robust configuration.

C. Baseline Systems

Model A: The original `KhaveeVerifier` of PyThaiNLP 5.0.1, instrumented verbatim from the vendored source, uses dictionary-based `subword_tokenize` and includes none of the main 6.0 refinements (Table I), such as the R_3 rule, karun handling, and true-final logic.

Model C: The `word_check` rhyme grader of KlonSuphap-LM [1] romanizes each wak with the TLTK G2P model and compares the romanized vowels and finals. Its two documented weaknesses are (i) a `replace_long_short` step that collapses long and short vowels and (ii) a mattra comparison that ignores the final consonant, both inflating recall at the cost of precision; TLTK failures reduce coverage.

IV. Dataset Construction and Augmentation

A. Gold-Standard Corpus Compilation

We collected five classical Thai narrative poems from the Vajirayana digital library [14]: *kʰoː.bùt* (โคบุตร), *kʰŭn tɕʰáːŋ kʰŭn pʰɛ̌ːn* (ขุนช้างขุนแผน), *pʰrá ʔàpʰaɪ mániː* (พระอภัยมณี), *nírâːt pʰuːkʰăo tʰɔːŋ* (นิราศภูเขาทอง), and *sùpʰaːsìt sɔ̌ːn jǐŋ* (สุภาษิตสอนหญิง). A cleaning pipeline normalized the text and split it into stanzas of four waks, discarding only chapter-opening fragments and scribal "๏" marks. Crucially, waks with ten or more syllables were retained; dropping them severed inter-stanza rhyme chains and caused an early corpus version to violate R_X at an apparent ∼72% rate, whereas on the complete corpus R_X holds at ∼96–97%. The final corpus comprises 191 files, 36,474 stanzas, 145,896 waks, and 145,702 gold rhyme checks; the gold is all-positive, because the classical poems are presumed to rhyme. Three stanzas in *pʰrá ʔàpʰaɪ mániː* (พระอภัยมณี) are incomplete in the source: a couplet is absent from the transcription, so the present half of each is excluded from stanza tiling rather than fabricated (their six waks are the only ones not counted above). Because the absent wak is the one that carried the cross-stanza rhyme, the stanza following each gap is treated as a fresh stanza with no predecessor (R_X not applicable), so the gap is never scored as a spurious R_X failure. The Thai Literature Corpora [15] cover similar material.

B. Targeted Data Augmentation

Recall over an all-positive corpus cannot measure precision, and easy positives cannot expose blind spots. We therefore augmented the gold with adversarial instances generated by an automated pipeline. Every instance is *standalone*: it is derived from a real stanza but only the target rule is changed, so exactly one rule is exercised while the others remain intact. Every negative is verified by the 6.0 oracle (`is_sumpus`) to be a genuine non-rhyme, and the whole set was audited in four independent passes in which reviewers spelled out the sara and mattra of every candidate (with vowel length made explicit) and double-checked pronunciations against the IPA. The audit removed 56 candidate words the oracle misreads—karun clusters it fails to silence and multi-syllable Pali words SSG keeps as single tokens.

Negatives (precision probes): The 10,000 negatives are drawn from eight corruption operators: trivial different-vowel-and-mattra swaps (C6, 300), same-mattra swaps (C0, 500), same-vowel swaps (C1, 500), short/long vowel swaps (C3, 3,100), liquid-final swaps (C4, 1,200), a systematic trap in which the 5.0.1 checker reads the candidate as rhyming with the target while 6.0 does not (C9, 2,800), leading-consonant words (C5, 600), and karun/*rua* (ฤ)/cluster traps (C2, 1,000). C3 and C9 dominate the mix because they are the operators that best discriminate the checkers.

Positives (recall probes): The 1,200 *tricky* positives swap the target syllable with a rhyming candidate that exercises normalization (*sàrà pliàn.rûːp/lót.rûp* สระเปลี่ยนรูป/ลดรูป), *rúa* (ฤ), karun, and true-final cases; all are oracle-confirmed rhymes.

TABLE II
GOLD RECALL (WHOLE-STANZA AND PER-RULE), 36,474 STANZAS.

System	Stanza	R_1	R_2	R_3	R_X
A (5.0.1)	74.0	86.3	89.5	N/A	89.4
B (6.0)	88.1	94.7	97.2	95.5	97.4
D_w2p	88.1	95.4	97.0	94.9	97.3
D_ssg	**89.2**	**95.9**	**97.4**	95.2	**97.8**
C (Kongfha)	85.5†	93.1	96.5	94.1	96.6

† coverage 97.6%; per-rule figures on the evaluated subset.

A further 423 *oracle-blind* positives are genuine rhymes the oracle labels as non-rhymes: the silent-*r* *pʰét* (เพชร) (true *e+kot class*, which the oracle reads with the long e: because the dead final carries no short-vowel mark), and the first-sara words *sàttru:* (ศัตรู), *kàsàttri:* (กษัตริ), and *kàsàttri:* (กษัตริย์) (SSG keeps each as one token, so `check_sara` hears only the first vowel). For these the gold label is the *linguistic* truth, not the oracle verdict: a checker that hears the real rhyme receives credit, while the oracle itself (Model B) is charged a false negative—"the oracle that is wrong loses points."

V. EVALUATION AND RESULTS

A. Experimental Setup

We report precision, recall, and F1 with Wilson 95% confidence intervals, coverage, and a conservative drop-as-fail variant for Model C. Because every augmentation instance is standalone and tests exactly one rule, the per-rule prediction is the clean metric; a whole-stanza aggregate over the augmentation is only well-defined for Model B (the oracle). Model A has no R_3 and is reported as N/A there. All checkers were instrumented from verbatim vendored sources (the 5.0.1 and 6.0 cores), with Model C run under Python 3.12 through a persistent pool of ten TLTK workers (the full augmentation pass takes about 40 minutes; A/B/D are parallelized and finish in under a minute).

B. Gold-Corpus Recall

Table II reports whole-stanza and per-rule recall over the 36,474 gold stanzas. The proposed family dominates: D_ssg reaches 89.2% stanza recall and B 88.1%, versus 74.0% for Model A. Model C reaches 85.5% but evaluates only 97.6% of the corpus (TLTK failures concentrate in the archaic *kʰŭn tɕʰáːŋ kʰŭn pʰɛ́ːn*).

C. Augmentation: Precision, Recall, and F1

Table III summarizes the augmentation-only results (pooled per rule). Three key findings stand out.

First, Model B is a perfect oracle on the negatives: zero false positives across all 10,000 (100% precision by construction). Its recall is limited only by the oracle-blind positives it cannot see (73.9%).

Second, Model D recovers most of those blind spots: D_w2p reaches 92.1% recall and D_ssg 87.0%, with precision 83.4% and 86.8%, giving the best augmentation F1 (87.5% for D_w2p). Table IV shows the mechanism: D_w2p hears 405 of the 423 oracle-blind rhymes (95.7%), D_ssg 288 (68.1%), B none.

TABLE III
AUGMENTATION-ONLY RESULTS (POOLED PER RULE, 11,623 INSTANCES). FP = FALSE POSITIVES ON 10,000 NEGATIVES; FN-POS = COMBINED FALSE NEGATIVES ON 1,623 POSITIVES.

System	P	R	F1	FP	FN-pos
A (5.0.1)	28.4	62.7†	39.1	1,950	848
B (6.0)	**100.0**	73.9	85.0	**0**	423
D_w2p	83.4	**92.1**	**87.5**	298	128
D_ssg	86.8	87.0	86.9	215	211
C (Kongfha)	30.4	87.1†	45.1	3,107	266

†R and F1 count only the rules each checker implements (A has no R_3, so its 380 R_3-targeting positives are excluded). FN-pos counts every positive without a true verdict, including R_3-N/A and dropped ones, so it exceeds the FN used in R for A and C.

TABLE IV
ORACLE-BLIND PROBE AND ERROR ANALYSIS (FN-POS = TRICKY + ORACLE-BLIND).

	recall (%)		FN-pos		FP	
System	tricky	oracle-blind	total	=oracle-blind	total	C3/C9
A	57.0	21.5	848	332	1,950	162/1,774
B	**100**	0.0	423	423	**0**	0/0
D_w2p	90.8	**95.7**	**128**	18	298	135/103
D_ssg	93.7	68.1	211	135	215	119/48
C	85.3	78.7	266	90	3,107	2,702/267

Third, the baselines invert the precision/recall trade-off. Model C is recall-strong (87.1% on the augmentation, 78.7% even on oracle-blind rhymes, because its G2P romanization hears the real sound) but makes 3,107 false positives (2,702 from the C3 short/long vowel swap, which its `replace_long_short` collapses), for only 30.4% precision. Model A is weak on both sides (28.4% precision, 62.7% recall), with 1,774 of 1,950 false positives from the C9 old-accept trap. Per rule, B keeps 100% precision on all four rules (R_1 recall lowest at 54.5%, R_X highest at 98.0%), C's precision collapses on every rule (24–35%), and D_ssg stays above 75% precision and 82% recall throughout.

D. Oracle-Blind Probe and Error Analysis

Table IV reports the oracle-blind probe with the combined false-negative accounting (tricky + oracle-blind) and the FP breakdown by the two most informative operators. The combined FN-pos column is the headline: B carries 423 false negatives (all oracle-blind), A carries 848, and D carries only 128 (D_w2p) or 211 (D_ssg). C's precision collapse concentrates on the short/long vowel swap (C3) and A's on the old-accept trap (C9).

E. Discussion

The experiments yield four main findings. (1) **The proposed rules are precise.** B never false-accepts on oracle-verified negatives, and B, D_w2p, and D_ssg all exceed 99% merged precision (A 95%, C 92%). (2) **The override dictionary recovers the oracle's blind spots.** The gap between D_w2p

and D_ssg on oracle-blind recall (95.7% versus 68.1%) is the pronunciation-knowledge contribution: the overrides read เพชร as *pʰét* and split ศัตรู and กษัตริย์ into their true syllables *sàt.truː* and *kà.sàt.triː*. A limitation is that each entry forces a single canonical pronunciation, although many surface forms are heteronyms whose reading depends on context, and adjacent syllables are often merged in delivery so that a rhyme lands. The entries are handcrafted from manually verified Word2Phrase (w2p) readings; mining a public gold-standard G2P corpus to expand the dictionary is left for future work. (3) **C trades precision for recall on exactly the vowel-length distinction** the rule-based family enforces (78.7% oracle-blind recall, but 30.4% precision). (4) **The oracle itself has a blind spot** that only the oracle-blind probe exposes: a purely oracle-supervised evaluation would report B as flawless. On the merged gold+augmentation set, F1 favors D_ssg (93.8%), D_w2p (93.3%), and B (93.2%), with C (88.0%) and A (82.6%) behind.

VI. Applications and Future Work

A. Educational Impact

The verifier is deployed as an interactive web tool for teachers and students studying Klon poetry: it segments the entered poem into syllables, highlights rhyme positions, spells out the sara and mattra of every syllable, and explains each error in terms of the four rhyme rules. The rule-based feedback is reproducible and hallucination-free. Future work includes classroom evaluations and pronunciation guidance for irregular words.

B. Foundation for Generative AI

The same determinism makes the verifier a suitable reward signal for fine-tuning small, locally run LLMs. It mirrors the RL step of the KlonSuphap-LM pipeline [1], but with a grader (Model B or D) substantially more precise and faster than the G2P-based grader used there (Model C), particularly on vowel length. Fine-tuning a generator with this reward, scaling the override dictionary from a public gold-standard G2P corpus, and handling heteronyms and on-the-fly syllable merging are future work, alongside decoding-time constrained sampling [13].

VII. Conclusion

We presented a hybrid rule-based approach to Thai Klon-Paed rhyme verification. An improved rule-based verifier (Model B), now the default `KhaveeVerifier` in PyThaiNLP main and to ship in 6.0, implements all four canonical rhyme rules with robust karun, true-final, and sara handling and reaches 100% precision on curated negatives. A dictionary-enhanced variant (Klon8Checker, Model D) adds a gold-standard G2P override dictionary of over 4,000 entries and recovers 95.7% of the oracle's own blind spots. On 36,474 gold stanzas and 11,623 adversarial instances, the proposed family raises stanza recall from 74.0% to 89.2% (D_ssg) with near-perfect precision, exposing the blind spots of both the G2P-romanization checker and the rule-based oracle. The verifier is released as an explainable educational tool and provides a deterministic reward signal for RL-based generation; its integration into PyThaiNLP makes the improvement available to the entire Thai NLP ecosystem.

Acknowledgment

We thank the Vajirayana Digital Library [14], the PyThaiNLP maintainers for merging the `KhaveeVerifier` improvements, and the TLTK and KlonSuphap-LM team.

References

[1] K. Yingyingseree, "KlonSuphap-LM: GPT-2 for Thai Klon-Paed," Hugging Face model, 2024. [Online]. Available: https://huggingface.co/Kongfha/KlonSuphap-LM

[2] PyThaiNLP Contributors, "PyThaiNLP: Thai natural language processing in Python," 2024. [Online]. Available: https://github.com/PyThaiNLP/pythainlp

[3] T. Kittinaradorn, et al., "DeepCut: A Thai word tokenization library using deep neural network," 2019. [Online]. Available: https://github.com/rkcosmos/deepcut

[4] P. Chormai, P. Prasertsom, and A. T. Rutherford, "AttaCut: A fast and accurate neural Thai word segmenter," arXiv preprint arXiv:1911.07056, 2019.

[5] P. Chormai, P. Prasertsom, J. Cheevaprawatdomrong, and A. T. Rutherford, "Syllable-based neural Thai word segmentation," in Proc. 28th International Conference on Computational Linguistics (COLING), 2020.

[6] W. Aroonmanakun, "TLTK: Thai language toolkit," [Online]. Available: https://pypi.org/project/tltk/

[7] P. Charoenpornsawat and T. Schultz, "Example-based grapheme-to-phoneme conversion for Thai," in Proc. Interspeech, 2006, doi: 10.21437/interspeech.2006-260.

[8] S. Saychum, S. Kongyoung, A. Rugchatjaroen, P. Chootrakool, S. Kasuriya, and C. Wutiwiwatchai, "Efficient Thai grapheme-to-phoneme conversion using CRF-based joint sequence modeling," in Proc. Interspeech, 2016, pp. 1462–1466, doi: 10.21437/interspeech.2016-621.

[9] A. Rugchatjaroen, S. Saychum, S. Kongyoung, P. Chootrakool, S. Kasuriya, and C. Wutiwiwatchai, "Efficient two-stage processing for joint sequence model-based Thai grapheme-to-phoneme conversion," Speech Communication, vol. 106, pp. 105–111, 2019, doi: 10.1016/j.specom.2018.12.003.

[10] P. Suksanguan, S. Waijanya, and N. Promrit, "The extraction of beautiful sound patterns from Sunthorn Phu's poem using machine learning technique and internal rhyme rule," Int. J. Adv. Intell. Inform., vol. 7, no. 2, pp. 198–210, 2021, doi: 10.26555/ijain.v7i2.613.

[11] K. Yingyingseree, "PhraAphaiManee-LM," Hugging Face model, 2023. [Online]. Available: https://huggingface.co/Kongfha/PhraAphaiManee-LM

[12] L. Ouyang, J. Wu, X. Jiang, et al., "Training language models to follow instructions with human feedback," in NeurIPS, 2022.

[13] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without fine-tuning," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[14] Vajirayana Digital Library. [Online]. Available: https://vajirayana.org

[15] A. Rutherford, "Thai literature corpora (TLC)," [Online]. Available: https://attapol.github.io/tlc.html